

Подготовка к защите лабораторных работ

Парадигмы ООП, SOLID, виртуальные методы и Q&A по работам №0–№5
Курс «Мультипарадигмальное программирование»

Документ построен как «шпаргалка для защиты». В первой части — теоретическая основа (парадигмы ООП, принципы SOLID, виртуальные методы) с привязкой к конкретному коду из репозитория `mp-sequence-lab`. Дальше — серия вопросов и ответов по каждой из шести лабораторных, включая «коварные» вопросы по дизайну, асимптотике и нюансам OpenMP. Каждый ответ сформулирован так, чтобы на защите хватило одной-двух произнесённых вслух фраз.

Содержание

№	Раздел
1	Парадигмы ООП и их проявления в коде
2	Принципы SOLID на примерах из репозитория
3	Виртуальные методы: зачем, когда обязательны, когда нет
4	Q&A по лаб. №0 — Sequence (Array/List)
5	Q&A по лаб. №1 — Алгоритмы сортировки
6	Q&A по лаб. №2 — Алгоритмы поиска
7	Q&A по лаб. №3 — OpenMP intro
8	Q&A по лаб. №4 — Релаксация Якоби 3D
9	Q&A по лаб. №5 — Параллельное умножение матриц
10	Сквозные «коварные» вопросы по всему циклу
11	Финальная шпаргалка

1. Парадигмы ООП и их проявления в коде

Под «парадигмами ООП» обычно понимают четыре опоры объектной парадигмы. Все четыре в репозитории проявлены явно, и удобно разбирать каждую на конкретных файлах.

1.1. Абстракция

Выделение существенного и сокрытие неважного. Класс описывается через что он умеет, а не как сделан. Базовый шаблон `Sequence<TElement>` (lab0/include/Sequence.h:3) задаёт интерфейс «последовательность» через `Get/Append/Prepend/InsertAt/Remove` — пользователю безразлично, лежит ли внутри `std::vector` или `std::list`. Тот же принцип в `ISorter<T>` (lab1/include/ISorter.h:7) — «сортировщик — это нечто, что умеет `Sort` и `GetName`».

1.2. Инкапсуляция

Данные скрыты за `private`, доступ — через методы; инварианты охраняются изнутри. Примеры: приватный `std::vector<TElement>` `data` в `ArraySequence` (lab0/include/ArraySequence.h:11) с приватным `checkIndex`; приватные `n_`, `a_`, `b_` в `Jacobi` (lab4/include/Jacobi.h:19); `data_/rows_/cols_` в `Matrix` (lab5/include/Matrix.h:64) с доступом только через `At(i, j)`.

1.3. Наследование

Производный класс получает интерфейс и/или реализацию от базового. В вашем коде это «наследование интерфейса»: `ArraySequence<T> : public Sequence<T>, ListSequence<T> : public Sequence<T>, BubbleSorter / InsertionSorter / QuickSorter : public ISorter<T>`. База абстрактная, каждый наследник обязан реализовать чисто виртуальные методы.

1.4. Полиморфизм

Один и тот же вызов через указатель/ссылку на базу даёт разное поведение в зависимости от реального типа объекта. У вас это работает в `MeasureSortTime(ISorter<T>& sorter, ...)` (lab1/src/main.cpp:23) — функция не знает, какая именно сортировка стоит за `sorter`, выбор делается в рантайме через `vtable`. Кроме рантайм-полиморфизма, у вас используется параметрический полиморфизм через шаблоны `template<typename T>` — это уже generic-парадигма поверх ООП.

1.5. Сводка по проявлению парадигм в каждой лабораторной

Лаба	Доминирующие парадигмы	Где это видно
lab0	ООП + generic	Sequence + ArraySequence/ListSequence, шаблон TElement
lab1	ООП + generic + Strategy	ISorter + 3 наследника, шаблон T, шаблонный MeasureSortTime
lab2	generic + функциональная	Шаблоны Key/Value, лямбда-геттер <code>std::function<Key(const T&)></code>
lab3	Параллельная (data-parallel)	OpenMP-директивы, свободные функции в namespace <code>omp_examples</code>
lab4	ООП-оболочка + параллельная	Класс Jacobi + <code>#pragma omp</code> внутри методов
lab5	ООП + процедурная + параллельная	Matrix как value-class, свободные функции в namespace <code>matmul</code>

2. Принципы SOLID на примерах из репозитория

SOLID — пять принципов проектирования объектных систем (Роберт Мартин). Идея проста: код, который удовлетворяет SOLID, проще менять и расширять без переписывания смежных модулей. Ниже каждый принцип привязан к конкретному классу из ваших работ.

2.1. S — Single Responsibility

Класс делает ровно одно дело. `BubbleSorter` сортирует пузырьком, `BinaryTree` хранит и ищет в BST, `Jacobi` считает релаксацию, `Matrix` представляет матрицу и доступ к её элементам. В lab2 SRP особенно выпуклый: три алгоритма поиска разнесены по трём отдельным классам — каждый отвечает только за свой подход.

2.2. O — Open / Closed

Открыт для расширения, закрыт для модификации. Чтобы добавить новую сортировку в lab1, не нужно менять `ISorter`, `MeasureSortTime`, `CompareSorters` или существующие сортировщики — достаточно написать новый класс-наследник `ISorter<T>`. Тот же приём в lab0 позволил бы добавить третью реализацию `Sequence<T>` (например, кольцевой буфер) без модификации существующих.

2.3. L — Liskov Substitution

Объект класса-наследника должен корректно подставляться вместо объекта базы. У вас `ArraySequence<int>` и `ListSequence<int>` ведут себя одинаково с точки зрения внешнего наблюдателя — отличаются только сложность операций. Тесты `runSequenceTest<TSequence>()` проходят для обоих, потому что LSP выполняется. Если бы, например, `ListSequence::Append` молча обрезал длину — LSP был бы нарушен, и общие тесты бы упали.

2.4. I — Interface Segregation

Лучше много узких интерфейсов, чем один «жирный». `ISorter` минимален (два метода). `Sequence` содержит только операции, которые имеют смысл для любой реализации. Нет «лишних» методов типа «сериализуй себя в JSON», которые одним наследникам нужны, а другим — нет. В lab2 принцип проявлен «от противного»: общего `ISearcher` там сознательно нет, потому что у трёх структур принципиально разные контракты — объединять их было бы натяжкой и противоречием ISP.

2.5. D — Dependency Inversion

Зависим от абстракций, а не от конкретных классов. Функция `MeasureSortTime(ISorter<T>& sorter, ...)` не знает про `QuickSorter`; код, написанный «против `Sequence<T>`», не знает про `std::vector/std::list`. Это и есть DIP в чистом виде — высокоуровневый код (бенчмарк, тесты) не привязан к низкоуровневой реализации.

Замечание. Важно понимать, что SOLID — это принципы дизайна объектных систем. К lab3 (свободные функции с OpenMP) и большей части lab4/lab5 (где ООП — лишь оболочка вокруг численного кода) SOLID применяется ограниченно. Там действуют другие принципы: cache locality, false sharing, NUMA-эффекты, ограничения Амдала/Густафсона. Это нормально: SOLID — не универсальный закон вселенной, а набор эвристик для конкретного типа задач.

3. Виртуальные методы: зачем, когда обязательны, когда нет

3.1. Что это технически

Виртуальный метод — это метод, выбор реализации которого происходит в рантайме по реальному типу объекта, а не в компайл-тайме по типу указателя/ссылки. C++ реализует это через `vtable`: у каждого объекта класса с виртуальными методами есть скрытый указатель на таблицу адресов функций. Каждый вызов виртуального метода — это разыменование указателя на `vtable`, выборка адреса метода и косвенный вызов.

3.2. Зачем они нужны

Чтобы можно было держать в общей переменной (или коллекции) объекты разных конкретных типов, обращаться к ним через общий интерфейс и получать разное поведение в зависимости от того, что лежит внутри. Канонический пример — `lab1`:

```
std::vector<std::unique_ptr<ISorter<int>>> sorters;
sorters.push_back(std::make_unique<BubbleSorter<int>>());
sorters.push_back(std::make_unique<QuickSorter<int>>());
for (auto& s : sorters) {
    auto result = s->Sort(data);    // вызывается нужный Sort
    std::cout << s->GetName();      // тоже виртуальный
}
```

Без `virtual` компилятор по типу `ISorter<int>*` всегда звал бы `ISorter<int>::Sort`, который у вас чисто абстрактный — программа бы даже не собралась. А если бы база была не абстрактной, вызывался бы метод базы, а не наследника. Виртуальность — это именно про «выбрать реализацию по реальному типу».

3.3. Виртуальный деструктор

Отдельная история, обязательная для любого класса, объекты которого могут удаляться через указатель на базу. Если в `Sequence` (`lab0/include/Sequence.h:6`) убрать `virtual ~Sequence() = default;`, то `Sequence<int>* p = new ArraySequence<int>(); delete p;` вызовет только деструктор базы, а внутренний `std::vector` утечёт. Это UB. Поэтому правило простое: есть `virtual`-методы → есть `virtual`-деструктор.

3.4. Когда они НЕ обязательны

Виртуальность стоит денег: указатель на `vtable` в каждом объекте, косвенный вызов вместо прямого, барьер для оптимизатора (нет инлайна). Поэтому добавлять её «на всякий случай» не нужно. Виртуальные методы не нужны в следующих случаях:

1. Нет наследников и/или нет вызова через базу. Класс не наследуют, либо его никто не держит через указатель на базу. Пример из ваших работ: `Matrix`, `BinaryTree`, `HashTable`, `Jacobi`, `Student` — у всех нет ни одного виртуального метода, и правильно.
2. Полиморфизм статический, через шаблоны. Разные варианты выбираются на этапе компиляции, и роль `virtual` играет параметр шаблона. `HashTable<Key, Value>` сам по себе уже полиморфен через `Key/Value` — никаких баз для него не нужно.
3. Горячий вычислительный код. `Matrix::At(i, j)` вызывается миллиарды раз в умножении матриц. Виртуальный вызов с косвенностью и без инлайна замедлил бы код в

разы.

4. `final`-классы. Если класс наследуется, но дальше никто не переопределяет методы, их можно пометить `final` — компилятор знает, что виртуальности по факту нет, и девиртуализирует вызовы.

5. CRTP / статический `dispatch`. Когда хочется общего интерфейса и нулевого оверхеда, используют `curiously recurring template pattern`. Это причина, по которой авторы STL не делают `std::vector` виртуальным.

3.5. Где конкретно в ваших работах `virtual` нужен, а где нет

Файл / класс	virtual?	Причина
lab0/Sequence	Да, все методы и деструктор	Полиморфизм через <code>Sequence<T>*</code> — основа задачи
lab0/ArraySequence, ListSequence	Да, override	Реализуют абстрактные методы базы
lab1/ISorter	Да, все методы и деструктор	Полиморфизм через <code>ISorter<T>&</code> в <code>MeasureSortTime</code>
lab1/*Sorter	Да, override	Реализуют абстрактные методы базы
lab2/BinaryTree	Нет	Нет семейства подклассов, вызовы напрямую
lab2/HashTable	Нет	Нет семейства подклассов
lab2/BinarySearch	Нет, это свободная функция	Не класс вообще
lab3/Examples	Нет, свободные функции	Обобщение
lab4/Jacobi	Нет	Один класс, нет наследников, горячий код
lab5/Matrix	Нет	Value-class, нет наследников; <code>At()</code> обязан инлайниться
lab5/matmul::*	Нет, свободные функции	Не классы

3.6. Короткая шпаргалка для защиты

- Нужны (lab0 Sequence, lab1 ISorter) — есть семейство наследников + код, работающий через указатель/ссылку на базу + желание подменять реализацию в рантайме.
- Не нужны и вреден оверхед (lab2 структуры поиска, lab3 свободные функции, lab4 Jacobi, lab5 Matrix) — нет наследников, либо выбор реализации делается на этапе компиляции, либо это горячий вычислительный код.
- Можно ли в lab1 убрать `virtual`? Да, если переписать `MeasureSortTime` как `template<typename Sorter>` — статический `dispatch`. Но тогда потеряется возможность хранить разнотипные сортировщики в одной коллекции, а это именно то, что демонстрирует lab1.

4. Q&A; по лаб. №0 — Sequence (Array / List)

В. Какие парадигмы здесь применены?

О. Объектно-ориентированная (классы, наследование, инкапсуляция, рантайм-полиморфизм через `Sequence<T>*`) и обобщённое (generic) программирование через `template<typename T>Element`. В тестах дополнительно используется функциональный приём — шаблонная `runSequenceTest<TSequence>()`, одна и та же логика для двух разных типов.

В. В чём заключаются принципы SOLID на примере вашей Sequence?

О. S — Sequence отвечает только за контракт «последовательность», `ArraySequence` — за реализацию через `vector`, `ListSequence` — через `list`. О — добавить третью реализацию можно, не трогая базу и существующие наследники. L — `ArraySequence` и `ListSequence` подставимы вместо `Sequence` без сюрпризов (одни и те же тесты проходят на обоих). I — интерфейс минимальный, без «жирных» методов. D — потребитель зависит от шаблонного типа, а не от конкретного класса.

В. Зачем `virtual ~Sequence() = default`?

О. Если объект наследника удаляется через указатель на базу (`Sequence<int>* p = new ArraySequence<int>(); delete p;`), без виртуального деструктора отработает только деструктор базы, и внутренний `vector/list` внутри наследника не уничтожится. Это UB и течь ресурсов.

В. Зачем = 0 (чисто виртуальные методы)?

О. Это делает `Sequence` абстрактным классом: его нельзя инстанцировать (`Sequence<int> s;` не компилируется), и каждый наследник обязан реализовать все эти методы. Компилятор сам ловит ошибку «забыл реализовать».

В. Почему методы помечены `override`?

О. `override` — это проверка от компилятора: если в базе нет такого виртуального метода с такой сигнатурой, будет ошибка компиляции. Без `override` опечатка (`Apend` вместо `Append`, или промах с `const`) молча создала бы новый невиртуальный метод, и полиморфизм был бы сломан без единого предупреждения.

В. Почему `Get(i)` в `ListSequence` — это $O(i)$, а в `ArraySequence` — $O(1)$?

О. В `std::vector` доступ по индексу — арифметика по адресу (база + $i \cdot \text{sizeof}(T)$). В `std::list` (двусвязный список) индекса нет, и `std::advance(it, index)` буквально проходит по ссылкам i раз. Это фундаментальное свойство структуры, его никаким наследованием не «починить».

В. Почему `GetSubsequence` возвращает конкретный тип, а не `Sequence`?

О. Через `Sequence` нельзя — абстрактный класс, объект по значению не создашь. Возвращать `Sequence<T>*` можно, но тогда вызывающая сторона должна `delete`-ить. Возврат по значению конкретного типа удобнее и не теряет тип. Платой становится «не-полиморфность» подпоследовательности: её тип знает вызывающий. Поэтому `GetSubsequence` сознательно не включён в базовый интерфейс.

В. Где здесь обязателен `virtual`, а где нет?

О. Обязателен: все методы Sequence и деструктор — иначе при работе через Sequence<T>* вызовутся пустые методы базы. Не обязателен: GetSubsequence намеренно не в базе, потому что возвращает «свой» тип; если бы он был в базе, пришлось бы возвращать указатель.

5. Q&A; по лаб. №1 — Алгоритмы сортировки

В. Зачем понадобился ISorter, если можно было звать сортировку напрямую?

О. Чтобы в `MeasureSortTime(ISorter<T>& sorter, ...)` (`lab1/src/main.cpp:23`) хранить любой сортировщик в переменной одного типа и единообразно его таймить. Это ОСР: добавляем новый сорт — `MeasureSortTime` не меняется. И это DIP: бенчмарк зависит от `ISorter`, а не от `QuickSorter`.

В. Какой паттерн проектирования вы использовали?

О. Strategy из GoF: «алгоритм — это объект, который можно подменять в рантайме». `ISorter` — интерфейс стратегии, `BubbleSorter/InsertionSorter/QuickSorter` — конкретные стратегии. `MeasureSortTime` — клиент, который работает с любой стратегией.

В. Какие парадигмы тут смешаны?

О. ООП через `ISorter` + наследование; обобщённое программирование через `template<typename T>`; немного процедурного — `MeasureSortTime` это свободная функция, а не метод; и функциональный приём — функции принимают и возвращают объекты-«стратегии».

В. Зачем Sort принимает const ArraySequence&, а внутри делает result = sequence?

О. Контракт «вход не модифицируется» — сортировка возвращает новую отсортированную последовательность, оригинал не трогается. `const ArraySequence<T>&` это документирует, и компилятор проверяет. Копия делается локально и уже она сортируется в памяти результата.

В. Почему Sort не const-метод, как GetName?

О. Технически мог бы быть, потому что состояние самого сортировщика не меняется. Но обычно резервируют возможность, что в наследнике появится накопительное состояние — счётчик сравнений, профилировщик. `GetName` — это «чистая функция от типа», а `Sort` — нет.

В. Можно ли в lab1 обойтись без виртуальных методов?

О. Да, если переписать `MeasureSortTime` как `template<typename Sorter> long long MeasureSortTime(Sorter& sorter, ...)` — тогда вызов `sorter.Sort(...)` разрешится статически, без `vtable`. Это даже быстрее. Но теряется возможность хранить разнотипные сортировщики в одной коллекции `vector<ISorter<int>*>`, а это в `lab1` именно демонстрируется. Поэтому виртуальные методы — осознанный выбор в пользу гибкости в рантайме.

В. Почему QuickSort с pivot по центру лучше, чем pivot = arr[left]?

О. На уже отсортированном (или обратно отсортированном) массиве выбор `arr[left]` даёт худший случай $O(n^2)$: `pivot` каждый раз экстремум, разбиение получается $1 + (n-1)$. Центральный `pivot` на отсортированных данных даёт сбалансированное разбиение, сложность остаётся $O(n \log n)$. На случайных данных разницы практически нет.

В. Почему BubbleSort у вас намеренно «глупый» (без early-exit)?

О. Чтобы получить честный верхний край асимптотики $O(n^2)$ и заметную разницу с `QuickSort` на больших n . Если добавить `early-exit` «если за проход не было обменов — массив отсортирован», то на уже отсортированных данных будет $O(n)$, и таблица сравнений

перестанет показывать разницу.

В. Стабилен ли каждый из ваших алгоритмов?

О. Bubble и Insertion — да: они не переставляют равные элементы. QuickSort у вас — нет: при разбиении с `pivot = arr[mid]` равные элементы могут перепрыгнуть друг через друга. Если требуется стабильность — QuickSort не подходит, нужен Merge Sort.

В. Какие особые случаи покрывают тесты?

О. Пустая последовательность, один элемент, уже отсортированная, обратно отсортированная, последовательность с дубликатами. Эти пять кейсов прогоняются через `TestSorter<T>` для всех трёх сортировщиков.

6. Q&A; по лаб. №2 — Алгоритмы поиска

В. Почему здесь нет общего интерфейса `ISearcher`, как был `ISorter` в lab1?

О. Потому что у трёх структур принципиально разные контракты: `BinarySearch` работает с уже отсортированным `std::vector<T>` и принимает геттер ключа; `BinaryTree` сам хранит пары; `HashTable` хранит пары и требует хэшируемого ключа. Объединять их в один интерфейс — натяжка ради натяжки, ISP против этого. Поэтому ООП-полиморфизм здесь не нужен, и `virtual` нигде нет.

В. Какая парадигма доминирует в lab2?

О. Обобщённое программирование через шаблоны `template<typename Key, typename Value>` плюс функциональное через `std::function<Key(const T&)>` в `BinarySearch.h:6` — лямбда-геттер позволяет искать по любому полю объекта, не переписывая алгоритм. ООП есть, но «номинальное»: классы как контейнеры данных + методов, без наследования.

В. Что не так с «магическим -1» как «не найдено»?

О. Это работает, только пока `Value` — целое и `-1` гарантированно не валиден. Стоит появиться `Value = string` или `Value = unsigned` — код развалится либо логически (`-1` валиден), либо синтаксически. Каноничное решение — `std::optional<Value>`, либо `bool Find(Key, Value& out)`, либо итератор `end()`. На защите это самый частый вопрос.

В. Чем $O(\log n)$ BST отличается от $O(\log n)$ бинарного поиска?

О. Бинарный поиск требует отсортированный массив — это $O(n \log n)$ подготовки. Зато потом `Find` — настоящий $O(\log n)$ с хорошей локальностью (соседние индексы в кеше). BST строится за $O(n \log n)$ в среднем, но `Find` работает с указателями: каждая нода в случайном месте памяти, плохая локальность. Плюс ваше BST несбалансированное — на отсортированном вставленном потоке вырождается в список и даёт $O(n)$. Если данные статичны — берём бинарный поиск; если часто вставляем/удаляем — BST (а лучше — сбалансированное).

В. Почему хэш-таблица с цепочками, а не открытой адресацией?

О. Цепочки проще реализовать (`vector<list<pair>>`) и устойчивы к высокому коэффициенту заполнения. Минус — в каждой ячейке `list` со своим оверхедом на ноды и плохой локальностью. Открытая адресация компактнее в памяти, но требует `rehash` и обработки удалений через `tombstones`.

В. В чём «фишка» лямбды-геттера в `BinarySearch`?

О. Алгоритм перестаёт зависеть от типа данных. Один и тот же шаблон ищет по любому полю: `[](const Student& s){ return s.age; }, [](const Student& s){ return s.group; }, [](const Student& s){ return s.name; }`. Это паттерн «инъекция извлечения ключа», тот же подход у `std::sort` с компаратором.

В. Минусы вашей конкретной реализации хэш-таблицы?

О. 1. `capacity = 100` захардкожен, нет автоматического `rehash` при росте `load factor`. 2. `100` — не простое число; для не-uniform ключей это может увеличить коллизии (лучше `101`). 3. `std::hash<int>` в `libstdc++` — `identity` (возвращает само число); `modulo` по неудачному `capacity` даёт предсказуемое распределение. 4. «-1 как не найдено».

В. Виртуальные методы где-то нужны здесь?

О. Нет. Семейства подклассов нет ни у одной структуры. Если бы хотелось подменять «бэкэнд поиска» в рантайме (ISearcher<Key,Value> с реализациями BSTSearcher, HashSearcher) — тогда нужны. Но это уже другая постановка задачи.

7. Q&A; по лаб. №3 — OpenMP intro

В. Какая парадигма здесь главная?

О. Параллельное (data-parallel) программирование через директивы OpenMP. ООП в lab3 нет — все примеры это свободные функции в namespace `omp_examples`. Это сознательный выбор: добавлять классы там, где данные «плоские» и алгоритм линейный, — лишняя сложность без пользы.

В. Что делает `#pragma omp parallel`?

О. Создаёт fork-join: при входе в блок мастер-поток порождает команду из `omp_get_num_threads()` потоков, каждый исполняет тело блока; на закрывающей скобке — неявный барьер, все потоки сходятся, остаётся только мастер. В Example 1 это используется для классического «Hello from thread N».

В. Что делает `reduction(+:sum)` в Example 2?

О. У каждого потока создаётся приватная копия `sum`, инициализированная нейтральным элементом операции (для `+` это 0). Поток считает свою часть произведения в свою копию. На выходе из параллельного региона компилятор автоматически суммирует все копии в общую `sum`. Без `reduction` несколько потоков одновременно делали бы `sum += ...` — это гонка данных и недетерминированный результат.

В. Почему в Example 2 нет гонки в `a[i]*b[i]`, но есть в `sum`?

О. Потому что каждый поток пишет в свой индекс `i` (`#pragma omp parallel for` разрезает диапазон без пересечений). А `sum` одна на всех — её обновление требует синхронизации. Базовое правило: гонка возникает только при пересекающихся записях в одну ячейку.

В. Что демонстрирует Example 3 с глобальными `g_sum`, `g_a`, `g_b`?

О. Что в OpenMP можно работать с общими данными и применять `#pragma omp for reduction(+:g_sum)` внутри функции, вызванной из `#pragma omp parallel`. В production так делать плохо — глобальные переменные и «магическое» зацепление через namespace. Это иллюстрация из методички, а не образец дизайна.

В. Что такое `#pragma omp sections`?

О. Альтернатива `for`: вместо «разрежь цикл на куски» это «вот тебе явные блоки, раздай по одному потоку». Используется, когда работа неоднородная и автоматически не делится. У вас две `section`-и считают разные половины массива `c`. `nowait` снимает барьер на закрывающей скобке `sections` — поток, закончивший раньше, идёт дальше, а не ждёт второго.

В. Зачем `critical` в Example 5, если в умножении матрицы на вектор нет гонок?

О. Гонки в самом вычислении (`c[i] += A[i][j] * b[j]`) действительно нет — каждый поток пишет в свой `i`. `critical` здесь нужен только для `printf`: без него вывод разных потоков смешивался бы внутри строки. То есть `critical` стоит вокруг I/O, а не вокруг арифметики. Это любимый вопрос на защите.

В. Можно ли в Example 5 заменить `critical` на `omp atomic`?

О. Нет. `atomic` работает только для одного простого RMW-обновления (`x += y`, `x++`), а `printf` — это вызов функции, его `atomic` не покрывает. Для произвольного блока — только `critical`.

или `ordered`.

В. Что задаёт `default(none)` в Example 1?

О. Запрещает неявную классификацию переменных как `shared/private`. Все используемые переменные нужно явно перечислить. Это защита от ошибки «забыл сделать `private` и получил гонку». В обучении полезно, в боевом коде — почти обязательно.

В. Чем `omp_set_num_threads` отличается от `num_threads()-clause`?

О. `omp_set_num_threads(N)` меняет глобальное значение по умолчанию для всех последующих `parallel`-регионов. `num_threads(N)` в директиве — только для одного `parallel`-региона. Для библиотечного кода безопаснее второй вариант: вызывающая среда не пострадает.

8. Q&A; по лаб. №4 — Релаксация Якоби 3D

В. Почему 3D-куб лежит в одном `std::vector`, а не как `vector<>>`?

О. Линейный layout даёт непрерывность в памяти и предсказуемый доступ для процессорного prefetch. `vector<vector<vector<>>>` — это массив указателей на массивы указателей: каждая внутренняя строка лежит где попало в куче, кеш промахивается на каждой смене i или j . На 3D-сетке 256^3 это легко даёт 5–10× разницу в скорости даже без OpenMP.

В. Зачем `Idx(i,j,k) = (i*N + j)*N + k`?

О. Это row-major индексация: соседи по k лежат рядом (+1), соседи по j — на расстоянии N , соседи по i — на расстоянии N^2 . Внутренний цикл по k бежит по непрерывной памяти, что идеально для кеша L1/L2.

В. Почему параллелится только внешний цикл по i ?

О. Этого достаточно. На $N = 128$ диапазон $i = 1..N-2 = 126$ разрезается на 2–8 потоков без проблем. Параллелить и j через `collapse(2)` можно, но это даст эффект только при очень маленьких N , когда $N-2$ меньше числа потоков. Плюс `collapse` ломает локальность кеша.

В. Почему нет гонок в `RelaxParallel`?

О. Потому что в итерации Якоби чтение идёт из A , запись — в B . Шаблон чтения соседей $(i\pm 1, j, k)$, $(i, j\pm 1, k)$, $(i, j, k\pm 1)$ пересекается между потоками, но это пересечение по чтению — оно безопасно. Если бы вы делали Gauss-Seidel (обновление in-place в A), параллелизация была бы куда сложнее (нужны цвета шахматной доски, wavefront-схема).

В. Почему `ResidParallel` использует «локальный `max + critical`», а не `reduction(max:eps)`?

О. По комментарию в README — для совместимости со старыми компиляторами и стабильной поддержкой. Паттерн «у каждого потока свой `localEps`, в конце `critical` обновляет глобальный `eps`» — это ручная редукция, эквивалентная по результату, но переносимая. С OpenMP 3.1+ можно и `reduction(max:eps)`, но на учебном коде ручной вариант показывает, как именно устроена редукция изнутри.

В. Почему здесь нет ни одного `virtual`-метода?

О. Класс `Jacobi` не образует семейства подклассов. Нет нужды подставлять «другой `Jacobi`» через указатель на базу. Виртуальный вызов в горячем коде релаксации был бы серьёзным замедлением — `vtable`-индирекция мешает инлайну.

В. Можно ли было сделать `Jacobi` абстрактным с двумя наследниками `SerialJacobi` и `ParallelJacobi`?

О. Да, и это было бы «правильнее» с точки зрения ООП — добавить новую стратегию (GPU, MPI) можно было бы без модификации старого. Но в текущей постановке экономика хуже выигрыша: код раздуется в 2 раза ради двух режимов — перебор. Когда стратегий 3+, тогда стоит выделять интерфейс.

В. Закон Амдала — что он даёт здесь?

О. Если доля последовательного кода s (инициализация, проверка ϵ ps), а параллельного $1-s$ ($\text{relax} + \text{resid}$), то предел ускорения $S(p) \leq 1 / (s + (1-s)/p)$. На малых N оверхеды $\text{fork/join} +$ ручная редукция дают s ощутимым, и на 2 потоках ускорение меньше 2. На больших N (256^3) — почти линейное ускорение, потому что s стремится к нулю.

В. Зачем `schedule(static)`, а не `dynamic`?

О. Все итерации i имеют одинаковую трудоёмкость (внутри один и тот же $(N-2)^2$ объём работы). `static` без оверхеда раздаёт по равным блокам. `dynamic` платит за каждый запрос блока — это оправдано, когда итерации сильно разной длины (не наш случай).

В. Зачем Release-сборка по умолчанию?

О. В `CMakeLists.txt` явно выставлен `CMAKE_BUILD_TYPE=Release`, если не задан другой. Без оптимизаций (`-O0`) разница между `serial` и `parallel` «съедается» накладными расходами OpenMP — это ключевой момент при анализе замеров. На `-O0` ускорения почти не будет.

9. Q&A; по лаб. №5 — Параллельное умножение матриц

В. Почему Matrix — плоский класс без виртуальных методов?

О. Это «value-class», ровно как `std::vector` или `std::string`: представляет данные + операции с ними. Виртуальность была бы overhead без пользы. И инлайн `At(i, j)` критичен для скорости — vtable сломала бы всю производительность.

В. Зачем два варианта параллельного умножения?

О. По индивидуальному варианту задачи: Способ 1 — число потоков задаётся параметром, размерность не обязана быть кратной числу потоков. Способ 2 — число потоков не задаётся, берётся из OpenMP по умолчанию (`OMP_NUM_THREADS` или количество ядер). Это требование задачи — продемонстрировать оба сценария.

В. В чём разница между `num_threads(threads)` и `omp_set_num_threads(threads)`?

О. `num_threads()` в директиве — локально для одного parallel-региона, не меняет глобальное состояние. `omp_set_num_threads()` — глобально, влияет на все последующие parallel-блоки. Для функции, которую могут звать из разных контекстов, `num_threads()` безопаснее: вызывающая среда не страдает.

В. Что если N не делится на число потоков?

О. `schedule(static)` сам корректно режет диапазон. При $N=7$, $threads=3$ получится $[0..2]$, $[3..5]$, $[6..6]$ — последний блок короче, никакой ручной арифметики не нужно. Это тестируется в `matmul_tests`: 7×7 на 3, 4 и 8 потоках.

В. Почему здесь не нужно ни `critical`, ни `atomic`?

О. Каждый поток пишет только в свою строку `s.At(i, *)`. Пересечений нет → гонок нет → синхронизация не нужна. Это идеальный случай для OpenMP. Если бы умножали наоборот (внутренний цикл по i), пришлось бы вводить `atomic` или редукцию по s .

В. Почему порядок циклов именно `i,j,p`, а не `i,p,j`?

О. Для последовательной версии порядок i,k,j часто быстрее: `a.At(i,k)` инвариантна по j , и внутренний цикл по j читает `b.At(k,j)` последовательно (row-major). Ваш порядок i,j,p оптимизирует доступ к `a.At(i,p)`, но `b.At(p,j)` идёт «вертикально» — каждая итерация прыгает на строку. Для $N \leq 1024$ это терпимо, но на больших размерах i,k,j побеждает за счёт лучшего кеша.

В. Почему `ApproxEqual` с абсолютным допуском $1e-6$?

О. Сравнение `double` побитово после арифметики нельзя — накапливаются ошибки округления. Абсолютный $1e-6$ ок для входных значений $[0, 10]$ и $N \leq 1024$: максимальная величина результата $\approx N \cdot 10 \cdot 10 = 10^5$, относительная погрешность 10^{-11} . Если бы значения были миллионы — нужен был бы относительный допуск $|a-b| < \text{eps} \cdot \max(|a|, |b|)$.

В. Закон Амдала или Густафсон — что важнее тут?

О. Амдал говорит «при фиксированной задаче ускорение ограничено долей последовательного кода». Густафсон — «если с ростом ядер мы и задачу увеличиваем, ускорение почти линейное». Для матричного умножения с ростом N доля параллельной части растёт как $O(N^3)$, а overheadы OpenMP — как $O(1)$ или $O(p)$. Поэтому на больших N

Густафсон показывает картину лучше, чем Амдал.

В. Виртуальные методы где-то нужны в lab5?

О. Нет. `Matrix` не наследуется, функции `Multiply*` — свободные функции в namespace `matmul`. Можно было бы сделать `IMatrixMultiplier` с тремя наследниками, но это разумно только если хочется сравнивать их единообразно, как в `lab1`. У вас `bench` делает это вручную через `TimeMillis(lambda)` — этого достаточно.

В. Зачем `FillRandom` принимает `seed`?

О. Для воспроизводимости: один и тот же `seed` даёт одни и те же матрицы, и `bench` всегда работает с теми же данными. Это позволяет сравнивать запуски на разных машинах и в разные дни — без `seed` погрешность измерения смешивалась бы с разницей входов.

10. Сквозные «коварные» вопросы по всему циклу

В. Почему вы используете шаблоны, а не `void*` / наследование от `Object`?

О. Шаблоны дают статическую типизацию (компилятор ловит ошибку «положил `int` туда, где ждали `string`») и отсутствие индирекции (никакой `vtable`, всё инлайнится). Цена — каждая инстанция генерирует свой код, бинарник распухает. Для контейнеров и алгоритмов это правильный выбор; для GUI/плагинов, где нужна динамика — наследование.

В. Когда `virtual` нужен, а когда нет — короткая шпаргалка по всем 6 лабам?

О. Нужен (lab0 Sequence, lab1 ISorter) — есть семейство наследников + код, работающий через указатель/ссылку на базу + желание подменять реализацию в рантайме. Не нужен и вреден (lab2 структуры поиска, lab3 свободные функции, lab4 Jacobi, lab5 Matrix) — нет наследников, либо выбор реализации делается на этапе компиляции, либо это горячий вычислительный код.

В. Парадигмы по лабам — где какая?

О. lab0, lab1 — ООП + обобщённое программирование. lab2 — обобщённое + функциональное (лямбды-геттеры). lab3 — параллельное (data-parallel via OpenMP), почти без ООП. lab4 — ООП-оболочка вокруг параллельного ядра. lab5 — комбинация: ООП (Matrix) + процедурное (свободные функции) + параллельное (OpenMP). Это ровно то, ради чего курс называется «мультипарадигмальное программирование».

В. SOLID применим ко всем работам одинаково?

О. Нет. lab0/lab1 — почти учебник по SOLID. В lab3 и lab4 SOLID почти неприменим, потому что код не объектный: там работают другие принципы (locality, false sharing, NUMA, амдалевские пределы). Это нормально — SOLID о дизайне объектных систем.

В. Где у вас потенциальные «дырки», которые могут спросить?

О. 1. lab2 — «-1 как не найдено» не обобщается; перейти на `std::optional`. 2. lab3 Example 3 — глобальные переменные в анонимном namespace; анти-паттерн вне учебной демонстрации. 3. lab4 ResidParallel — critical вокруг скаляра можно заменить на `reduction(max:eps)` (OpenMP 3.1+). 4. lab5 порядок циклов — `i,j,p` менее cache-friendly, чем `i,k,j`; на больших `N` стоило бы переупорядочить. 5. lab1 QuickSort — нестабилен; на массивах из одинаковых элементов деградирует к $O(n^2)$. На защите могут спросить про «трёхпутевое разбиение» (Dutch National Flag).

В. Что такое RAII, и где это видно в вашем коде?

О. RAII (Resource Acquisition Is Initialization) — ресурсы захватываются в конструкторе и автоматически освобождаются в деструкторе. У вас RAII работает «через STL»: `std::vector` в Matrix, ArraySequence, Jacobi сами освобождают память при разрушении объекта. Вы нигде не пишете `delete` явно — это и есть RAII. Исключение — BinaryTree, где `new Node` нигде не парный `delete`: есть утечка при разрушении дерева (нет деструктора, чистящего узлы). На защите это могут спросить.

В. Что такое гонка данных (data race) и где у вас она могла бы быть?

О. Гонка данных — две и более операции с одной и той же ячейкой памяти из разных потоков, и хотя бы одна — запись, и нет синхронизации. Поведение программы становится

UB. У вас гонки могли бы быть: 1. lab3 Example 2 — если убрать reduction и оставить `sum += a[i]*b[i]`. 2. lab4 ResidParallel — если читать-писать общий `eps` без `critical`. Везде, где их могло быть, вы вставили либо `reduction`, либо `critical`.

В. Что такое false sharing и есть ли он у вас?

О. False sharing — это ситуация, когда два потока пишут в разные переменные, но эти переменные лежат в одной кеш-линии (обычно 64 байта). Кеш-линия становится «hot» — постоянно invalid-ируется между ядрами, и производительность падает. У вас false sharing потенциально мог бы быть в lab5, если бы потоки писали в соседние элементы одной строки `s`. Но вы параллелите по строкам — каждый поток пишет в свою строку из `cols·8` байт, при `cols ≥ 8` строки не пересекаются по кеш-линиям.

11. Финальная шпаргалка для защиты

11.1. Парадигмы ООП — четыре опоры

- Абстракция — описание через «что умеет», а не «как сделан». Пример: `Sequence<T>`, `ISorter<T>`.
- Инкапсуляция — `private` поля, доступ через методы. Пример: `Matrix::data_`, `Jacobi::a_`, `b_`.
- Наследование — наследник получает интерфейс/реализацию. Пример: `ArraySequence/ListSequence` от `Sequence`.
- Полиморфизм — один вызов через базу, разное поведение в рантайме. Пример: `ISorter<T>&` в `MeasureSortTime`. Плюс параметрический полиморфизм через шаблоны.

11.2. SOLID — пять принципов

- S — один класс, одна ответственность.
- O — расширяемо без модификации старого.
- L — наследник заменяет базу без сюрпризов.
- I — много узких интерфейсов лучше одного «жирного».
- D — зависим от абстракций, не от конкретики.

11.3. Виртуальные методы — правила

- Нужны, если есть наследники + работа через указатель/ссылку на базу + желание выбирать реализацию в рантайме.
- Виртуальный деструктор обязателен в любом классе с виртуальными методами.
- `override` — обязательная страховка от опечаток.
- Не нужны: нет наследников, или `dispatch` статический через шаблоны, или это горячий вычислительный код.
- В ваших работах: есть в `lab0` `Sequence` и `lab1` `ISorter`; нет и правильно в `lab2`, `lab3`, `lab4`, `lab5`.

11.4. OpenMP — ключевые директивы

- `#pragma omp parallel` — `fork-join`, порождает команду потоков.
- `#pragma omp for` — распределение итераций цикла по потокам.
- `reduction(op:var)` — приватные копии + автоматическое комбинирование.
- `critical` — взаимное исключение для блока.
- `atomic` — взаимное исключение для одной RMW-операции (быстрее `critical`).
- `schedule(static)` — равные блоки, `dynamic` — по запросу.
- `nowait` — снять неявный барьер.
- `num_threads(N)` — локально для региона; `omp_set_num_threads(N)` — глобально.

11.5. Закон Амдала

При фиксированной задаче с долей последовательного кода s ускорение на p потоках ограничено $S(p) \leq 1 / (s + (1-s)/p)$. Чем меньше s , тем ближе к линейному ускорению. На маленьких размерах задачи `fork/join` съедает выигрыш — параллельная версия может оказаться медленнее последовательной.

11.6. Если спросят неожиданное

Не паникуйте. Главное — не утверждать того, в чём не уверены. Хороший ответ на «не знаю» звучит как «здесь я бы посмотрел X , потому что подозреваю Y » — это показывает, что у вас есть способ дойти до ответа, а не просто белое пятно.